



Beyond Monotonic Progress: Retry-Supervised Value Learning for Robot Imitation

Xinyao Qin^{*+1}, Junjie Lu^{*+2}, Kaixin Wang³, Chuheng Zhang³, Sinjae Kang⁴, Kimin Lee^{4,5}, Min Xu²,
Bin Liang¹, Jun Yang¹, and Li Zhao³

¹Tsinghua University, ²University of Technology Sydney, ³Microsoft Research Asia, ⁴KAIST, ⁵Config

* Equal contribution

† Work done at Microsoft Research Asia

Human demonstrations for robot imitation learning often contain mistakes and corrective behaviors, such as imprecise grasps, object misalignment, unstable contact, and repeated attempts. While these segments are commonly treated as noisy or suboptimal data, they provide valuable evidence about when execution deviates from a desirable path and how task feasibility can be restored. However, existing reward and value models often rely on monotonic progress assumptions, which capture coarse task advancement but may overlook local execution errors and corrective behaviors in imperfect demonstrations. In this work, we propose ReTVL (**Re**Try-Supervised Value Learning), a framework for learning mistake-sensitive value functions from mixed-quality robot demonstrations by leveraging retry events as sparse supervision. ReTVL captures the local degradation-and-recovery structure around mistakes by combining global progress calibration with local pairwise preference learning induced by sparsely annotated retry keypoints. The learned value model is then used to reweight demonstration chunks for downstream behavior cloning, reducing the influence of harmful execution errors while preserving useful corrective behaviors. Experiments on real-robot manipulation tasks show that ReTVL produces more fine-grained value estimates than progress-based baselines and improves imitation learning from imperfect demonstrations.

Keywords: Robot Learning, Value Learning, Reward Modeling

demo: <https://youtu.be/6aF6QrPg2To>

1 Introduction

Vision-Language-Action (VLA) models have significantly improved the capabilities of general-purpose robotic policies in recent years [3, 5, 7, 8, 11, 20, 24, 30, 42]. By integrating visual and language understanding into action generation, these models have demonstrated great generality across diverse manipulation tasks [15, 21, 27, 39, 41, 50]. At the same time, applying VLAs to downstream real-world tasks still relies heavily on human demonstration data, which can be noisy and imperfect in practice [22, 25, 43, 49]. Such data often contains hesitation, imprecise manipulations, and repeated attempts, yet these noisy or suboptimal segments are often ignored or simply discarded. In this work, we study the problem of value learning from such imperfect demonstration data.

Recent work learns value or reward functions from robot data by estimating task progress [3, 13, 14, 26, 28, 29, 33, 40, 47]. These methods typically assume that task progress can be represented as a monotonically increasing scalar signal along a trajectory, which fails to capture intermediate imperfect segments such as mistakes and retries. For example, as illustrated at the top of Figure 1, a teleoperator may overshoot or make mistakes and later correct their behavior. Modeling such trajectories as monotonic progress can therefore lead to inaccurate value functions, as shown by the line plot in Figure 1. Consequently, using these inaccurate value estimates for downstream robot imitation learning can degrade the performance of the resulting policy.

Addressing this issue requires explicitly accounting for retry events during value learning. Such retry events are common in robot demonstration data and often incur little additional labeling cost, yet they are typically ignored or discarded. An intuitive observation is that a retry event often indicates that execution has deviated from a desirable path and is subsequently brought back through corrective behavior. Rather than treating such retries as noisy or low-quality data, we leverage them as sparse

Example trajectory with **retries** (task: *Stack Blocks*)

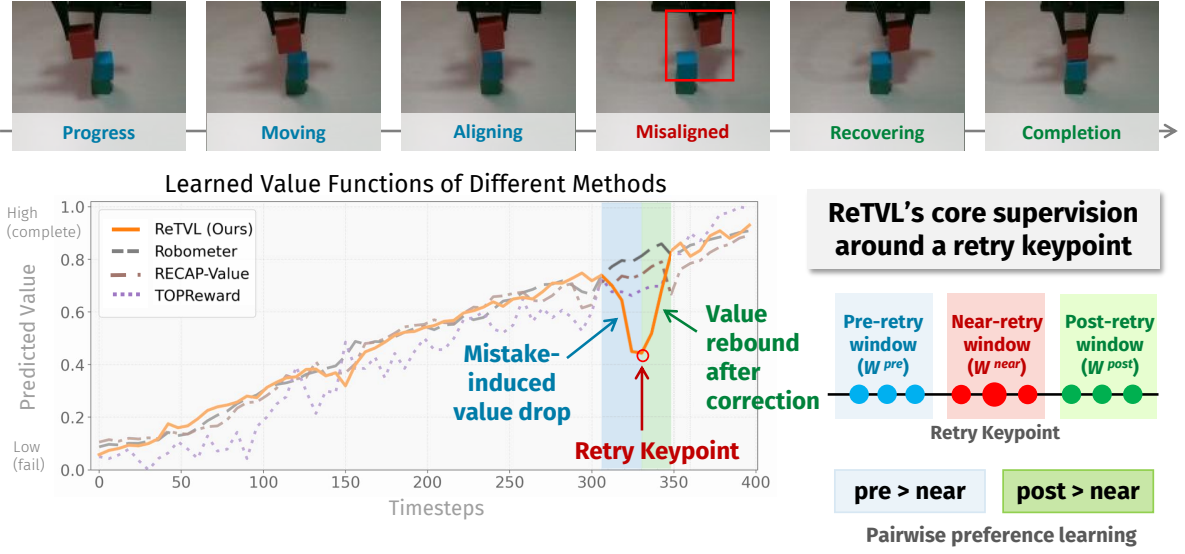


Figure 1: **ReTVL turns retry events into pairwise value supervision.** Progress-based value models may overlook subtle execution errors and assign overly smooth increasing values. ReTVL uses retry keypoints to learn local value drops before correction and rebounds after recovery, enabling better identification of harmful and corrective trajectory segments.

supervisory signals for value learning. Specifically, the temporal neighborhood around a retry reveals a local value structure: value should decrease as execution drifts toward a degraded state that requires correction, and increase as corrective behavior restores task feasibility. These retry events therefore induce local pairwise preferences, allowing the value model to capture value drops and rebounds around subtle execution errors without requiring dense frame-level progress labels.

Based on this insight, we propose ReTVL, a new value learning framework that leverages retry events as sparse supervision. Specifically, ReTVL constructs state pairs around retry events and introduces a preference-based loss to train the value function. We find that this design is critical for accurate value estimation, whereas a naive strategy that simply penalizes retry events within conventional progress-regression-based value learning performs poorly. We evaluate ReTVL on four real-world robot manipulation tasks against three representative value estimation methods: TOPReward [14], Robometer [29], and RECAP-Value [3]. The results show that ReTVL accurately assigns lower values to harmful segments and higher values to useful corrective behaviors, capturing the value dynamics around retry events without collapsing into an uninformative monotonic trend. We further apply the learned value functions to downstream policy learning by reweighting action chunks from suboptimal demonstration datasets. Compared with standard behavior cloning and progress-based baselines, ReTVL achieves higher success rates across diverse manipulation tasks. Our contributions are summarized as follows:

- (1) We identify retry events as informative supervision for value learning, revealing local value drops and rebounds that are often hidden by coarse progress labels.
- (2) We propose ReTVL, which combines global progress supervision with local relative preferences to learn values that reflect both execution errors and corrective behaviors.
- (3) We show that ReTVL improves both value estimation and downstream policy learning on mixed-quality demonstrations, outperforming progress-based value models and their corresponding value-weighted policy learning baselines.

2 Related Works

2.1 Learned Value and Reward Models for Robot Learning

Learned reward and value models have been widely used to provide dense supervision for robot learning when task success is sparse or difficult to specify manually. Early work infers rewards from demonstrations through inverse reinforcement learning, but often faces scalability challenges in high-dimensional, long-horizon problems [1, 18, 35, 51]. Other work learns dense reward signals from human videos [12, 31], cross-embodiment videos [2], or language-image supervision [2, 32], enabling reward estimation for downstream manipulation tasks. More recent methods scale reward modeling with large robotics datasets and vision-language models, learning general-purpose reward or value models from progress labels [3, 26], success signals [28, 29], trajectory comparisons [29], or stage annotations [13, 40]. These learned values have been used for success-failure prediction, data filtering, and policy improvement in downstream tasks [3, 13]. While prior methods mainly capture global progress and success likelihood, they may fail to reflect subtle execution errors. Our method instead leverages retry-based supervision to learn local mistake-and-recovery dynamics.

2.2 Learning Policy from Suboptimal Demonstrations

Learning from suboptimal demonstrations has been widely studied to reduce the need for clean expert data. Prior imitation learning methods estimate demonstration quality through confidence scores [44], learned reliability or confidence estimation [49], distribution matching [23], or discriminator-based weighting [45], and use these signals to select or reweight useful demonstrations for imitation learning. Other work uses ranked demonstrations [9, 10] or preference feedback [17, 36, 38] to guide policy optimization beyond suboptimal demonstrators without manually specified rewards. A more closely related direction learns value functions and applies offline RL or weighted behavior cloning to prioritize useful actions during policy learning [3, 13, 19, 48]. Our method follows this weighted BC paradigm, but derives the weights from a retry-supervised value model, which more directly suppresses local mistake segments in mixed-quality demonstrations.

3 Method

3.1 Problem Setup

Value Function. Let $h_t = \mathbf{o}_{t-H+1:t}$ denote the history observation window of size H ending at time t . Given h_t and a task instruction ℓ , the value model estimates the current task progress as $V_\theta(h_t, \ell) \in [0, 1]$. In practice, we use a distributional value function that predicts a categorical distribution over K discretized progress bins for the stability and reliability of value estimation. Let $p_\theta(k | h_t, \ell)$ be the predicted probability of the k -th bin, and b_k denote the center value of that bin. We compute the scalar value as the expected progress under the predicted distribution:

$$V_\theta(h_t, \ell) = \sum_{k=0}^{K-1} b_k p_\theta(k | h_t, \ell). \quad (1)$$

Progress Supervision. To provide a global progress reference, we apply absolute progress supervision to frames outside retry neighborhoods. For a successful trajectory of length T , the target progress at timestep t is defined as $v_t^* = t/T \in [0, 1]$, which provides a coarse estimate of how far the execution has advanced. For failed trajectories, we assign $v_t^* = 0$ at terminal frames. Each target progress value v^* is discretized into a target bin b^* , and the model is trained to predict this bin with a cross-entropy loss [3]:

$$\mathcal{L}_{\text{abs}} = -\mathbb{E}_{(h_t, b_t^*)} \log p_\theta(b_t^* | h_t, \ell), \quad (2)$$

This objective provides a global progress signal by assigning lower values to earlier states and higher values to later states along the trajectory.

Data Annotations. We consider a demonstration dataset $\mathcal{D} = \{\tau_i\}_{i=1}^N$, where each trajectory τ_i is associated with a binary outcome label $y_i \in \{0, 1\}$ indicating task failure or success. For trajectories that contain retry behaviors, we additionally annotate a set of retry keypoints $\mathcal{R}_i = \{r_{i,j}\}_{j=1}^{M_i}$. Each keypoint

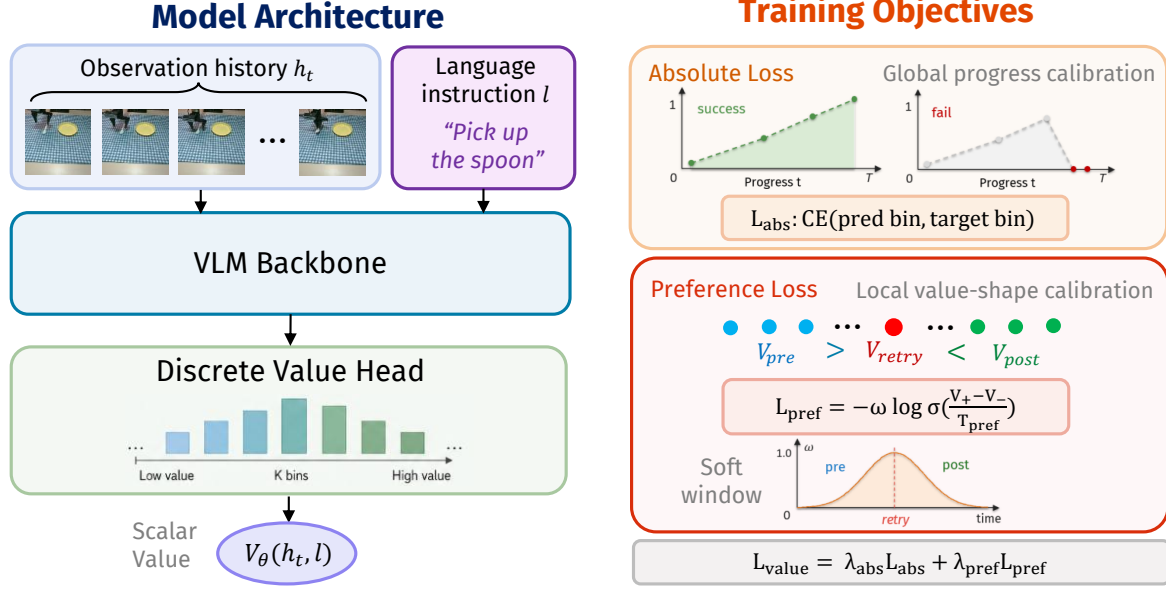


Figure 2: **ReTVL learns retry-sensitive value estimates from sparse retry annotations.** The model takes an observation history and language instruction as input, and predicts a scalar value through a VLM backbone and discrete value head. Training combines absolute progress calibration with retry-induced preference supervision, where values drop near retry states and rebound after recovery.

$r_{i,j}$ marks the start of the j -th corrective behavior in trajectory τ_i . Notably, we do not annotate the full retry segment such as the start of deviation or the completion of recovery. Instead, we only annotate the moment when corrective behavior starts. This choice makes the supervision more reliable and easier to obtain, since the full retry segment can be annotator-dependent, while the start of correction is usually easy to label during data collection or extracted from existing human-correction datasets [22, 46].

3.2 Pairwise Value Learning from Retry Events

Pair Construction. For each retry keypoint $r_{i,j}$, we construct preference pairs based on a simple but natural drop-and-rebound assumption: values should decrease before the correction starts and increase after effective corrective behavior begins. To this end, we divide the retry neighborhood into three regions:

$$\mathcal{W}_{i,j}^{\text{pre}} = \{t \mid r_{i,j} - \Delta_{\text{pre}} \leq t < r_{i,j} - \Delta_{\text{near}}\}, \quad (3)$$

$$\mathcal{W}_{i,j}^{\text{near}} = \{t \mid r_{i,j} - \Delta_{\text{near}} \leq t \leq r_{i,j} + \Delta_{\text{near}}\}, \quad (4)$$

$$\mathcal{W}_{i,j}^{\text{post}} = \{t \mid r_{i,j} + \Delta_{\text{near}} < t \leq r_{i,j} + \Delta_{\text{post}}\}. \quad (5)$$

Here, Δ_{near} defines a small neighborhood around the retry keypoint, while Δ_{pre} and Δ_{post} define the temporal ranges used for sampling before and after the retry, respectively. We then sample preference pairs (h^+, h^-) such that the retry neighborhood forms a local value minimum, as it corresponds to a degraded state where correction becomes necessary. On the pre-retry side, values are expected to be higher than those in the near-retry region and gradually decrease before the retry keypoint. On the post-retry side, values should increase over time and remain higher than those in the near-retry region. This construction provides local supervision for both the value drop before a retry and the value rebound after corrective behavior.

Soft-Window Weighting. The retry neighborhoods defined above are only approximate partitions of the local retry region and can be ambiguous in practice. Assigning equally strong preference constraints to all sampled pairs may therefore introduce noisy supervision, especially for pairs that rely on frames far from the retry keypoint or close to uncertain window boundaries. To mitigate this issue, we use a soft-window weighting scheme. For each sampled pair (h^+, h^-) around retry keypoint $r_{i,j}$, we compute

the weight using the endpoint farther from the retry keypoint, which corresponds to the preferred history h^+ under the drop-and-rebound assumption. Let t^+ denote the ending timestep of h^+ . We define its temporal distance to the retry keypoint as

$$d(h^+; r_{i,j}) = |t^+ - r_{i,j}|. \quad (6)$$

The soft weight is then defined as

$$w(h^+, h^-) = \exp(-d(h^+; r_{i,j})/\tau_w), \quad (7)$$

where τ_w controls the decay rate. This weighting assigns stronger supervision to pairs whose anchor is closer to the retry keypoint and smoothly downweights pairs whose anchor is farther away, making the preference loss less sensitive to imprecise retry-window boundaries.

Preference Loss. Given a sampled preference pair (h^+, h^-) , where h^+ should have a higher value than h^- , we optimize the soft-weighted pairwise logistic loss [6, 16]:

$$\mathcal{L}_{\text{pref}} = -\mathbb{E}_{(h^+, h^-)} w(h^+, h^-) \log \sigma \left(\frac{V_\theta(h^+, \ell) - V_\theta(h^-, \ell)}{T_{\text{pref}}} \right). \quad (8)$$

Here, T_{pref} is a temperature parameter that controls the sharpness of the preference comparison, $\sigma(\cdot)$ denotes the sigmoid function, and $w(h^+, h^-)$ is the soft-window weight. This loss enforces local value ordering around retry events, encouraging value drops before correction and value recovery after corrective behavior.

We then train the value model on a mixed-quality demonstration dataset combining absolute loss and preference loss. Absolute progress samples are drawn from all trajectories, except for frames inside retry neighborhoods where monotonic progress labels may be unreliable. In contrast, retry-based preference pairs are sampled from retry regions to provide local relative supervision. The two supervision sources are mixed with a fixed sampling ratio ρ_{pref} . The total training objective combines global progress calibration and local retry-based preference learning, balanced by λ_{abs} and λ_{pref} :

$$\mathcal{L}_{\text{value}} = \lambda_{\text{abs}} \mathcal{L}_{\text{abs}} + \lambda_{\text{pref}} \mathcal{L}_{\text{pref}}. \quad (9)$$

3.3 Value-Guided Behavior Cloning

We use our value model to guide policy learning from mixed-quality demonstrations to evaluate whether it brings practical improvements to downstream robot learning. This evaluation follows the weighted behavior cloning procedure from prior work [13, 34, 37]. For an action chunk starting at time t with stride Δ_a , we compute its weight from the predicted progress improvement and normalize the weight using offline dataset statistics:

$$r_t = V_\theta(h_{t+\Delta_a}, \ell) - V_\theta(h_t, \ell), \quad \alpha_t = \text{clip} \left(\frac{r_t - (\mu - 2\sigma)}{4\sigma + \epsilon}, 0, 1 \right), \quad (10)$$

where μ and σ are the offline mean and standard deviation of value improvements over the training dataset. The policy π_ψ is trained with a normalized weighted BC objective, which downweights chunks with low or negative predicted progress improvement while emphasizing chunks that move toward higher predicted task progress:

$$\mathcal{L}_{\text{wBC}}(\psi) = \frac{\sum_t \alpha_t \ell_{\text{BC}}(\pi_\psi(h_t), a_t)}{\sum_t \alpha_t + \epsilon}. \quad (11)$$

4 Experiments

Our experiments aim to study whether ReTVL can produce mistake-sensitive value estimates for subtle execution errors and thereby improve downstream policy learning. Specifically, we organize our experiments to answer the following questions:

(Q1) Value Evaluation: Does our value model better capture subtle execution errors and show value changes than progress-based reward or value models?

Table 1: Value evaluation across four real-world tasks. Global metrics measure trajectory-level success calibration, while local metrics measure retry-related mistake sensitivity.

Method	Global Metrics		Local Metrics			
	VOC $\uparrow$	S/F Det. $\uparrow$	Drop AUC $\uparrow$	Drop Prob. $\uparrow$	Pre > Retry $\uparrow$	Post > Retry $\uparrow$
TOPReward	0.903	0.844	0.264	0.612	0.329	0.917
Robometer	0.994	0.964	0.237	0.435	0.086	0.828
RECAP-Value	0.985	1.000	0.372	0.721	0.296	0.854
ReTVL	0.987	1.000	0.797	0.874	0.740	0.967

(Q2) Policy Learning: Given the same mixed-quality demonstration dataset, can our learned values identify more useful trajectory segments and improve downstream imitation learning?

(Q3) Ablation and Analysis: Which components of ReTVL are most important, and why does it improve value quality and policy performance?

Baselines. We compare ReTVL against the strongest available reward and value models that can provide dense scores for robot learning. These baselines cover three representative forms of dense supervision for robot learning: zero-shot VLM-based progress estimation, general-purpose robotic reward modeling, and progress-supervised value learning.

- **TOPReward** [14] directly prompts a pre-trained VLM to judge whether the task is completed and use the normalized probability of the “true” token as a progress score.
- **Robometer** [29] is a general-purpose robotic reward model trained on large-scale robot data. It uses trajectory rewinding and trajectory-level preferences to learn rewards for failed and suboptimal executions beyond progress supervision.
- **RECAP-Value** denotes our implementation of the value component in RECAP [3]. It uses Monte Carlo step-to-success targets to learn a progress value function with a discrete value head.

Tasks and Datasets. We evaluate our method on 4 real-robot manipulation tasks using a Piper robot arm, as illustrated in Appendix A. *Pick up Spoon* is a relatively simple pick-and-place task, while the other three tasks are long-horizon tasks covering different manipulation challenges: *Stack Blocks* requires fine-grained alignment when stacking three blocks, *Fold Towel* involves deformable-object manipulation, and *Open Drawer* requires interacting with an articulated object. For each task, we use 30 annotated trajectories to train the value model and evaluate the learned values on 20 held-out trajectories. The trained value model is then used to score a separate mixed-quality dataset for downstream policy learning, consisting of 200 trajectories per task except *Pick up Spoon*, which uses 80 trajectories.

4.1 Value Evaluation

We first evaluate whether ReTVL can achieve strong global value estimation while better capturing subtle execution errors than baselines. We report both global and local metrics in Table 1. The global metrics include Value-Order Correlation (VOC) on clean successful trajectories and Success-Fail Detection (S/F Det.). They evaluate whether the value model captures temporal progress in successful executions and distinguishes successful trajectories from failed ones, reflecting its basic ability to estimate global task progress and terminal success. The local metrics evaluate whether the value model produces a localized value drop around subtle execution errors and recovers after correction. Drop PR-AUC and Drop Probability measure the alignment and frequency of such drops near retry events, while Pre > Retry and Post > Retry measure whether pre-mistake and post-recovery states are valued higher than retry states. We provide formal definitions in Appendix B.

Table 1 shows that ReTVL achieves competitive global value estimation while substantially improving local mistake sensitivity. On global metrics, ReTVL obtains a VOC score of 0.987, close to the best score from Robometer, and achieves the best Success-Fail Detection score of 1.000, matching RECAP-Value. These results indicate that ReTVL retains the basic ability to model trajectory-level progress

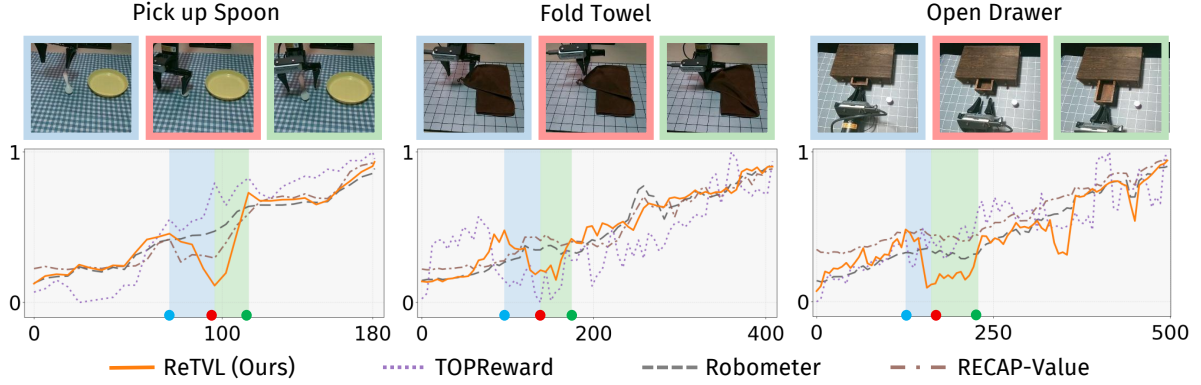


Figure 3: **Visualization of value evaluation.** We show value predictions on three other tasks beyond stack blocks. ReTVL captures local value drops around retry keypoints and rebounds after correction more clearly than progress-based baselines.

Task	Standard BC	RECAP -BC	ReTVL -BC
Pick up Spoon	60	65	85
Stack Blocks	45	80	95
Fold Towel	50	65	80
Open Drawer	10	40	60
Average	41	63	80

Table 2: Success rate (%) of the learned policies on 4 real-robot tasks, calculated from 20 trials per task.

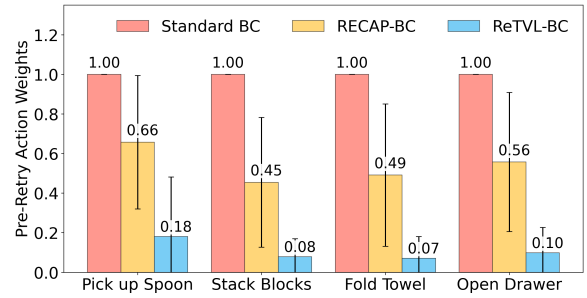


Figure 4: Average training weight assigned to annotated bad-action chunks in recovery trajectories. Lower is better.

and distinguish successful executions from failures. The main advantage of ReTVL lies in local retry-centered metrics. It achieves consistent improvements across all four local metrics, indicating that it more reliably captures retry-related local value changes. The improvement is especially large on Pre > Retry, where ReTVL reaches 0.740 while all baselines remain below 0.330. This suggests that existing methods often fail to assign lower values to bad actions or problematic states that trigger retries, as illustrated in Figure 3. Notably, although Robometer achieves the highest VOC score, its Pre > Retry score is the lowest, only 0.086, suggesting that strong global progress correlation does not necessarily imply sensitivity to local execution mistakes. In contrast, ReTVL induces the desired local value shape around mistakes, decreasing in degraded states and recovering after correction.

4.2 Policy Learning

Next, we evaluate whether the learned value model can improve downstream imitation learning from mixed-quality demonstrations. We compare **ReTVL-BC** against two baselines: **Standard BC**, which trains on all demonstrations with uniform weights, and **RECAP-BC**, which uses RECAP-Value to weight demonstration chunks. RECAP-Value is chosen because it provides the strongest local mistake detection among the baselines while maintaining competitive global value estimation in above Section 4.1. For both value-weighted methods, we split trajectories into action chunks and apply the same weighting rule from Section 3.3. The only difference is whether the weights are computed by RECAP-Value or our ReTVL model.

Table 2 shows that ReTVL-BC consistently improves downstream policy learning across all four real-robot tasks. Standard BC is sensitive to the quality of mixed demonstrations, achieving an average success rate of only 41.25%, as it imitates both clean successful actions and locally harmful actions from failed or retry trajectories. RECAP-BC improves the average success rate to 62.50% by assigning larger weights to higher-progress chunks, but its progress-based value signal is still less effective in separating useful corrections from locally mistaken actions. In contrast, ReTVL-BC achieves the best average success rate of

Table 3: Ablation study on held-out trajectories. We report representative global and local metrics averaged over four tasks.

Method	VOC $\uparrow$	S/F Det. $\uparrow$	Drop AUC $\uparrow$	Drop Prob. $\uparrow$	Pre > Retry $\uparrow$	Post > Retry $\uparrow$
ReTVL	0.987	1.000	0.797	0.874	0.740	0.967
w/o Preference Loss	0.994	1.000	0.510	0.836	0.486	0.742
w/o Soft Window	0.940	1.000	0.785	0.874	0.707	0.971
w/o Abs. Calibration	0.929	0.967	0.857	0.872	0.789	0.959

80.00%, improving over Standard BC by +38.75% and over RECAP-BC by +17.50%. These improvements suggest that retry-supervised values produce a better imitation-learning distribution by down-weighting mistake segments while emphasizing clean or recovery-relevant actions.

We further analyze the weights assigned to bad actions in Fig. 4, where bad actions are manually annotated on the held-out test set from Section 4.1. Standard BC assigns all actions a weight of 1.0. RECAP-BC reduces the average bad-action weight to 0.54, while ReTVL-BC further lowers it to 0.11, a 80% reduction over RECAP-BC. ReTVL also shows much smaller variance, suggesting that it consistently suppresses harmful actions rather than only down-weighting a few obvious mistakes.

4.3 Ablation and Analysis

We conduct ablation studies to identify which components of ReTVL contribute to mistake-sensitive value estimation. Table 3 first shows that the retry-induced preference loss is crucial for learning mistake-sensitive values. The **w/o Preference Loss** variant replaces our pairwise preference objective with an intuitive alternative: directly regressing to a shaped progress target with manually injected value drops around retry windows. Although this variant preserves strong global metrics, achieving even higher VOC (0.994) and the same Success-Fail Detection (1.000), it fails on local mistake-sensitive metrics. Drop AUC decreases sharply from 0.797 to 0.510, Pre > Retry falls from 0.740 to 0.486, and Post > Retry drops from 0.967 to 0.742. These results indicate that simply adding local drop penalties to a progress target does not reliably teach the model the relative ordering between mistaken and recovered states. In contrast, our preference loss directly supervises local comparisons around retry events, which is essential for capturing mistake-and-recovery structure.

The other two ablations reveal the complementary roles of soft windowing and absolute calibration. The **w/o Soft Window** variant removes the soft retry-window weighting and applies retry-related supervision uniformly. Its local metrics remain broadly comparable to the full model, but VOC decreases from 0.987 to 0.940. This suggests that hard retry supervision can create conflicts with the absolute progress targets, especially near the boundary between normal progress regions and retry neighborhoods. The **w/o Absolute Calibration** variant removes the absolute progress and success-failure calibration losses, leaving only retry-induced preference supervision. Although it can still learn local retry-centered ordering, its global value structure becomes weaker, with VOC dropping from 0.987 to 0.929 and Success-Fail Detection from 1.000 to 0.967. Overall, ReTVL benefits from all three components: preference supervision provides local mistake sensitivity, absolute calibration anchors the global value scale, and soft windowing stabilizes their interaction. Detailed visualizations of the ablation behaviors are provided in Appendix B.

5 Conclusion

We presented ReTVL, a retry-supervised value learning framework for robot imitation from mixed-quality demonstrations. By using sparsely annotated retry keypoints as local supervision anchors, ReTVL combines global progress calibration with retry-induced preference learning to capture both coarse task progress and local mistake-recovery structure. The learned values further improve downstream behavior cloning by down-weighting harmful mistake segments and emphasizing useful recovery behavior. Real-world manipulation experiments show that ReTVL improves local mistake sensitivity over progress-based baselines while maintaining competitive global value estimation, leading to better imitation learning from imperfect demonstrations. These results suggest that corrective behaviors are not merely noisy artifacts, but useful supervision for learning mistake-aware robot policies.

Limitations ReTVL still has several limitations. First, we assume that values around retry events follow a local degradation-and-recovery pattern. Although this is more flexible than a monotonic progress assumption, it may not cover all real-world correction patterns, such as exploratory retries or unsuccessful corrections. Second, we currently use the learned value model only for offline behavior cloning reweighting, and have not explored broader closed-loop policy improvement or online RL settings. Third, our experiments are conducted on a limited set of real-robot tasks rather than large-scale datasets, so broader evaluation is needed to assess generalization across more tasks, robots, and demonstration styles.

References

- [1] Abbeel, P. and Ng, A. Y. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 1, 2004.
- [2] Alakuijala, M., McLean, R., Woungang, I., Farsad, N., Kaski, S., Marttinen, P., and Yuan, K. Video-language critic: Transferable reward functions for language-conditioned robotics. *arXiv preprint arXiv:2405.19988*, 2024.
- [3] Amin, A., Aniceto, R., Balakrishna, A., Black, K., Conley, K., Connors, G., Darpinian, J., Dhabalia, K., DiCarlo, J., Driess, D., Equi, M., Esmail, A., Fang, Y., Finn, C., Glossop, C., Godden, T., Goryachev, I., Groom, L., Hancock, H., Hausman, K., Hussein, G., Ichter, B., Jakubczak, S., Jen, R., Jones, T., Katz, B., Ke, L., Kuchi, C., Lamb, M., LeBlanc, D., Levine, S., Li-Bell, A., Lu, Y., Mano, V., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Sharma, C., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Stoeckle, W., Swerdlow, A., Tanner, J., Torne, M., Vuong, Q., Walling, A., Wang, H., Williams, B., Yoo, S., Yu, L., Zhilinsky, U., and Zhou, Z. $\pi_{0.6}^*$: a vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- [4] Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [5] Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L. X., Tanner, J., Vuong, Q., Walling, A., Wang, H., and Zhilinsky, U. π_0 : A vision-language-action flow model for general robot control, 2026. URL <https://arxiv.org/abs/2410.24164>.
- [6] Bradley, R. A. and Terry, M. E. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- [7] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., Florence, P., Fu, C., Arenas, M. G., Gopalakrishnan, K., Han, K., Hausman, K., Herzog, A., Hsu, J., Ichter, B., Irpan, A., Joshi, N., Julian, R., Kalashnikov, D., Kuang, Y., Leal, I., Lee, L., Lee, T.-W. E., Levine, S., Lu, Y., Michalewski, H., Mordatch, I., Pertsch, K., Rao, K., Reymann, K., Ryoo, M., Salazar, G., Sanketi, P., Sermanet, P., Singh, J., Singh, A., Soricut, R., Tran, H., Vanhoucke, V., Vuong, Q., Wahid, A., Welker, S., Wohlhart, P., Wu, J., Xia, F., Xiao, T., Xu, P., Xu, S., Yu, T., and Zitkovich, B. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023. URL <https://arxiv.org/abs/2307.15818>.
- [8] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jackson, T., Jesmonth, S., Joshi, N. J., Julian, R., Kalashnikov, D., Kuang, Y., Leal, I., Lee, K.-H., Levine, S., Lu, Y., Malla, U., Manjunath, D., Mordatch, I., Nachum, O., Parada, C., Peralta, J., Perez, E., Pertsch, K., Quiambao, J., Rao, K., Ryoo, M., Salazar, G., Sanketi, P., Sayed, K., Singh, J., Sontakke, S., Stone, A., Tan, C., Tran, H., Vanhoucke, V., Vega, S., Vuong, Q., Xia, F., Xiao, T., Xu, P., Xu, S., Yu, T., and Zitkovich, B. Rt-1: Robotics transformer for real-world control at scale, 2023. URL <https://arxiv.org/abs/2212.06817>.
- [9] Brown, D. S., Goo, W., Nagarajan, P., and Niekum, S. Extrapolating beyond suboptimal demonstrations via inverse reinforcement learning from observations. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 783–792. PMLR, 2019.

- [10] Brown, D. S., Goo, W., and Niekum, S. Better-than-demonstrator imitation learning via automatically-ranked demonstrations. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pp. 330–359. PMLR, 2020.
- [11] Bu, Q., Cai, J., Chen, L., Cui, X., Ding, Y., Feng, S., Gao, S., He, X., Hu, X., Huang, X., et al. Agibot world colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.
- [12] Chen, A. S., Nair, S., and Finn, C. Learning generalizable robotic reward functions from “in-the-wild” human videos. In *Proceedings of Robotics: Science and Systems (RSS)*, 2021.
- [13] Chen, Q., Yu, J., Schwager, M., Abbeel, P., Shentu, F., and Wu, P. Sarm: Stage-aware reward modeling for long horizon robot manipulation. *arXiv preprint arXiv:2509.25358*, 2025.
- [14] Chen, S., Harrison, C., Lee, Y.-C., Yang, A. J., Ren, Z., Ratliff, L. J., Duan, J., Fox, D., and Krishna, R. Topreward: Token probabilities as hidden zero-shot rewards for robotics. *arXiv preprint arXiv:2602.19313*, 2026.
- [15] Chen, X., Wei, H., Zhang, P., Zhang, C., Wang, K., Guo, Y., Yang, R., Wang, Y., Xiao, X., Zhao, L., Chen, J., and Bian, J. villa-x: Enhancing latent action modeling in vision-language-action models, 2025. URL <https://arxiv.org/abs/2507.23682>.
- [16] Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [17] Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [18] Finn, C., Levine, S., and Abbeel, P. Guided cost learning: Deep inverse optimal control via policy optimization. In *International conference on machine learning*, pp. 49–58. PMLR, 2016.
- [19] Huang, W., Ren, P., Wang, J., Qi, Q., and Sun, H. Awr: Adaptive weighting regression for 3d hand pose estimation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 11061–11068, 2020.
- [20] Intelligence, P., Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Galliker, M. Y., Ghosh, D., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., LeBlanc, D., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Tanner, J., Vuong, Q., Walke, H., Walling, A., Wang, H., Yu, L., and Zhilinsky, U. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>.
- [21] Jiang, Y., Gupta, A., Zhang, Z., Wang, G., Dou, Y., Chen, Y., Fei-Fei, L., Anandkumar, A., Zhu, Y., and Fan, L. Vima: General robot manipulation with multimodal prompts, 2023. URL <https://arxiv.org/abs/2210.03094>.
- [22] Kelly, M., Sidrane, C., Driggs-Campbell, K., and Kochenderfer, M. J. HG-DAGger: Interactive imitation learning with human experts. In *Proceedings of the IEEE International Conference on Robotics and Automation*, pp. 8077–8083, 2019. doi: 10.1109/ICRA.2019.8793698.
- [23] Kim, G.-H., Seo, S., Lee, J., Jeon, W., Hwang, H., Yang, H., and Kim, K.-E. Demodice: Offline imitation learning with supplementary imperfect demonstrations. In *International Conference on Learning Representations*, 2022.
- [24] Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., and Finn, C. Openvla: An open-source vision-language-action model, 2024. URL <https://arxiv.org/abs/2406.09246>.
- [25] Laskey, M., Lee, J., Fox, R., Dragan, A., and Goldberg, K. Dart: Noise injection for robust imitation learning, 2017. URL <https://arxiv.org/abs/1703.09327>.
- [26] Lee, T., Wagenmaker, A., Pertsch, K., Liang, P., Levine, S., and Finn, C. Roboreward: General-purpose vision-language reward models for robotics. *arXiv preprint arXiv:2601.00675*, 2026.

- [27] Li, P., Chen, Y., Wu, H., Ma, X., Wu, X., Huang, Y., Wang, L., Kong, T., and Tan, T. Bridgevla: Input-output alignment for efficient 3d manipulation learning with vision-language models, 2025. URL <https://arxiv.org/abs/2506.07961>.
- [28] Li, Y., Ma, X., Xu, J., Cui, Y., Cui, Z., Han, Z., Huang, L., Kong, T., Liu, Y., Niu, H., et al. Gr-rl: Going dexterous and precise for long-horizon robotic manipulation. *arXiv preprint arXiv:2512.01801*, 2025.
- [29] Liang, A., Korkmaz, Y., Zhang, J., Hwang, M., Anwar, A., Kaushik, S., Shah, A., Huang, A. S., Zettlemoyer, L., Fox, D., Xiang, Y., Li, A., Bobu, A., Gupta, A., Tu, S., Biyik, E., and Zhang, J. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.
- [30] Liu, S., Wu, L., Li, B., Tan, H., Chen, H., Wang, Z., Xu, K., Su, H., and Zhu, J. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- [31] Ma, Y. J., Sodhani, S., Jayaraman, D., Bastani, O., Kumar, V., and Zhang, A. Vip: Towards universal visual reward and representation via value-implicit pre-training. *arXiv preprint arXiv:2210.00030*, 2022.
- [32] Ma, Y. J., Liang, W., Som, V., Kumar, V., Zhang, A., Bastani, O., and Jayaraman, D. Liv: Language-image representations and rewards for robotic control. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [33] Mao, Y., Yu, Z., Mao, W., Li, Y., Hu, Q., Lan, Z., Zhu, M., and Chen, H. Arm: Advantage reward modeling for long-horizon manipulation. *arXiv preprint arXiv:2604.03037*, 2026.
- [34] Nair, A., Gupta, A., Dalal, M., and Levine, S. Awac: Accelerating online reinforcement learning with offline datasets, 2021. URL <https://arxiv.org/abs/2006.09359>.
- [35] Ng, A. Y. and Russell, S. J. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning, ICML '00*, pp. 663–670, San Francisco, CA, USA, 2000. Morgan Kaufmann Publishers Inc. ISBN 1558607072.
- [36] Palan, M., Landolfi, N. C., Shevchuk, G., and Sadigh, D. Learning reward functions by integrating human demonstrations and preferences. In *Proceedings of Robotics: Science and Systems*, 2019.
- [37] Peng, X. B., Kumar, A., Zhang, G., and Levine, S. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning, 2019. URL <https://arxiv.org/abs/1910.00177>.
- [38] Sadigh, D., Dragan, A. D., Sastry, S. S., and Seshia, S. A. Active preference-based learning of reward functions. In *Proceedings of Robotics: Science and Systems*, 2017. doi: 10.15607/RSS.2017.XIII.053.
- [39] Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., Alibert, S., Cord, M., Wolf, T., and Cadene, R. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025. URL <https://arxiv.org/abs/2506.01844>.
- [40] Tan, H., Chen, S., Xu, Y., Wang, Z., Ji, Y., Chi, C., Lyu, Y., Zhao, Z., Chen, X., Co, P., et al. Robo-dopamine: General process reward modeling for high-precision robotic manipulation. *arXiv preprint arXiv:2512.23703*, 2025.
- [41] Team, O. M., Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., Luo, J., Tan, Y. L., Chen, L. Y., Sanketi, P., Vuong, Q., Xiao, T., Sadigh, D., Finn, C., and Levine, S. Octo: An open-source generalist robot policy, 2024. URL <https://arxiv.org/abs/2405.12213>.
- [42] Wen, J., Zhu, Y., Li, J., Tang, Z., Shen, C., and Feng, F. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025.
- [43] Wu, Y.-H., Charoenphakdee, N., Bao, H., Tangkaratt, V., and Sugiyama, M. Imitation learning from imperfect demonstration, 2019. URL <https://arxiv.org/abs/1901.09387>.
- [44] Wu, Y.-H., Charoenphakdee, N., Bao, H., Tangkaratt, V., and Sugiyama, M. Imitation learning from imperfect demonstration. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 6818–6827. PMLR, 2019.

- [45] Xu, H., Zhan, X., Yin, H., and Qin, H. Discriminator-weighted offline imitation learning from suboptimal demonstrations. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 24725–24742. PMLR, 2022.
- [46] Xu, X., Hou, Y., Liu, Z., and Song, S. Compliant residual dagger: Improving real-world contact-rich manipulation with human corrections. *Advances in Neural Information Processing Systems*, 38: 139559–139581, 2026.
- [47] Yang, J., Lin, K., Li, J., Zhang, W., Lin, T., Wu, L., Su, Z., Zhao, H., Zhang, Y.-Q., Chen, L., et al. Rise: Self-improving robot policy with compositional world model. *arXiv preprint arXiv:2602.11075*, 2026.
- [48] Yang, R., Wang, H., Liu, C., Yan, X., Wang, Y., Du, X., Yue, S., Liu, Y., Zhang, C., Qi, L., et al. Aloe: Action-level off-policy evaluation for vision-language-action model post-training. *arXiv preprint arXiv:2602.12691*, 2026.
- [49] Zhang, S., Cao, Z., Sadigh, D., and Sui, Y. Confidence-aware imitation learning from demonstrations with varying optimality. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- [50] Zhao, W., Ding, P., Zhang, M., Gong, Z., Bai, S., Zhao, H., and Wang, D. Vlas: Vision-language-action model with speech instructions for customized robot manipulation, 2025. URL <https://arxiv.org/abs/2502.13508>.
- [51] Ziebart, B. D., Maas, A. L., Bagnell, J. A., Dey, A. K., et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pp. 1433–1438. Chicago, IL, USA, 2008.

A Implementation Details

A.1 Value Model Training

Data preprocessing. All value models are trained and evaluated using the same local 5 Hz data protocol. The raw robot trajectories are recorded at 30 Hz and then downsampled by taking every sixth frame. We use this lower frame rate because adjacent 30 Hz frames typically show very small visual changes, which adds temporal redundancy and can make value estimation more sensitive to frame-level noise. Since the four tasks have substantially different horizons, we do not truncate trajectories to a fixed length after resampling. Instead, we keep all usable 5 Hz frames and construct fixed-length inputs later during sampling.

Each trajectory used for value-model training contains the task instruction, primary-view images, wrist-view images, the success/failure outcome, and retry keypoints. For the value-model test set, we additionally annotate bad-action start points and back-to-normal points, which are used only for analysis and evaluation. In all value-model training and evaluation, the model takes only the primary camera view as visual input. For value-model training and evaluation, the input at endpoint t is an 8-frame history window $h_t = o_{t-7:t}$. When $t < 7$, we left-pad the window by repeating the first frame. The model predicts the value of the last frame in the window, while the preceding frames serve as temporal context.

Architecture and hyperparameters. For a fair comparison, we use Robometer-4B [29] as the backbone for all trainable value models, which is pretrained on large-scale robotic data and follows the Qwen/Qwen3-VL-4B-Instruct [4] architecture. We attach a two-layer MLP value head on top of the vision-language hidden state. For history-window inputs, frames are fed as separate images in the multi-image input. The value head is applied only to the hidden representation of the last frame in the window. We fine-tune the backbone with LoRA and train the value head jointly with the LoRA parameters. All trainable methods use the same hyperparameters, summarized in Table 4.

For ReTVL, we instantiate the pre-retry, retry-centered, and post-recovery regions on the 5 Hz endpoint timeline as defined in Sec. 3.2. In each training step, the sampler draws one of the active pair types from four pair types with equal probability: pre-vs-near, near-vs-post, pre-vs-pre and post-vs-post. For pre-vs-near and near-vs-post pairs, the retry-centered window is treated as the lower-value side of the comparison. For pre-vs-pre and post-vs-post, we sample two endpoints from the same side of the retry keypoint and assign the lower value to the endpoint closer to the retry. Thus, values are encouraged to decrease as the trajectory approaches the retry from the pre-retry side and to increase as it moves away from the retry on the post-recovery side. We apply the same soft-window weighted pairwise preference loss described in Sec. 3.2. We use the same sampling parameters for all four tasks as summarized in Table 4. Algorithm 1 summarizes the overall training procedure.

Algorithm 1 Training Procedure for ReTVL

Require: Mixed-quality demonstration dataset $\mathcal{D}$, retry keypoints $\{\mathcal{R}_i\}$, value model V_θ

- 1: **for** each training iteration **do**
 - 2: Sample trajectories $\{\tau_i\} \sim \mathcal{D}$
 - 3: Sample absolute progress examples outside retry neighborhoods
 - 4: Compute absolute progress loss $\mathcal{L}_{\text{abs}}$ using Eq. 2
 - 5: Sample retry keypoints $r_{i,j} \in \mathcal{R}_i$ and construct $\mathcal{W}_{i,j}^{\text{pre}}, \mathcal{W}_{i,j}^{\text{near}}, \mathcal{W}_{i,j}^{\text{post}}$
 - 6: Sample preference pairs (h^+, h^-) and compute weights $w(h^+, h^-)$
 - 7: Compute preference loss $\mathcal{L}_{\text{pref}}$ using Eq. 8
 - 8: Update V_θ with $\mathcal{L}_{\text{value}} = \lambda_{\text{abs}}\mathcal{L}_{\text{abs}} + \lambda_{\text{pref}}\mathcal{L}_{\text{pref}}$
 - 9: **end for**
-

A.2 Downstream Weighted Behavior Cloning

For downstream policy learning, we use a flow-matching Vision-Language-Action policy with a π_0 -style backbone. The policy takes the task instruction, RGB observations, and robot proprioception as input, and predicts an action chunk. Given the chunk weight α_t computed by Eq. 10, we apply value-weighted

Table 4: Implementation hyperparameters used in our experiments.

Value Model Training		ReTVL Pair Sampling	
Hyperparameter	Value	Hyperparameter	Value
Input frame rate	5 Hz	Pre-retry window	$[r - 12, r - 2]$
History window	8 frames	Retry-centered window	$[r - 1, r + 1]$
MLP dims	[512, 512]	Post-recovery window	$[r + 2, r + 12]$
MLP activation	GELU	Soft-window temperature	$\tau_w = 6.0$
Distributional value bins	64	Preference temperature	$T_{\text{pref}} = 0.1$
MLP dropout	0.1	Abs. loss weight	$\lambda_{\text{abs}} = 1.0$
LoRA rank	32	Pref. loss weight	$\lambda_{\text{pref}} = 3.0$
LoRA alpha	64	Downstream Policy Training	
LoRA dropout	0.05	Learning rate	5×10^{-5}
Batch size	64	Training steps	20k
Learning rate	1×10^{-4}	Batch size	256
Scheduler	Cosine	Action chunk size	16
Training steps	500	Inference flow steps	10

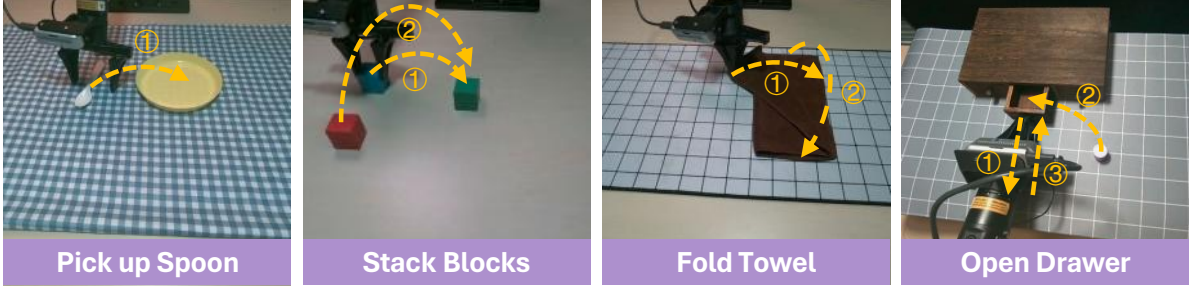


Figure 5: Real-world manipulation tasks used for policy evaluation.

behavior cloning directly to the flow-matching objective:

$$\mathcal{L}_{\text{wBC}} = \frac{\sum_t \alpha_t \|v_\psi(z_s, h_t, s) - u_s\|_2^2}{\sum_t \alpha_t + \epsilon},$$

where z_s is the interpolated noisy action chunk at flow time s , and u_s is the corresponding target velocity. Standard BC uses the same objective with uniform chunk weights. For ReTVL-BC and RECAP-BC, the policy architecture, training data, and optimization settings are identical; only the value model used to compute α_t differs. We fine-tune all policy parameters and use the same downstream training hyperparameters for all methods, as summarized in Table 4. Following the SARM-style weighting rule, we use task-specific thresholds κ : chunks with value improvement larger than κ are assigned weight 1. For each task, κ is chosen to roughly correspond to the 80th percentile of positive chunk improvements.

A.3 Robot Setup and Tasks

All real-robot experiments are conducted on an AgileX Piper robot arm. We collect demonstrations through leader-follower teleoperation. The setup includes one fixed third-person primary camera and one wrist-mounted camera. During policy execution, the policy takes the task instruction, RGB observations from both cameras, and the current robot proprioception as input. The policy outputs a 6-DoF end-effector pose command together with a continuous gripper command.

We evaluate four manipulation tasks, as illustrated in Fig. 5. In *Pick up Spoon*, the robot grasps a spoon and places it onto a plate. In *Stack Blocks*, the robot first stacks the blue block on top of the green block and then stacks the red block on top of the blue block. In *Fold Towel*, the robot folds a towel twice to form

a square-like shape. In *Open Drawer*, the robot opens a drawer, places a purple cylinder inside, and then closes the drawer.

Each policy is evaluated with 20 independent real-robot trials per task. Object positions are randomized before each trial. A trial is counted as successful only if the full task sequence is completed and the final task state satisfies the task-specific success criterion.

B Evaluation Metrics

We evaluate each value model on held-out trajectories that are disjoint from the annotated trajectories used for training. For each trajectory, we first evaluate the value model on all 5 Hz frames and then linearly interpolate the predicted values back to the raw-frame timeline. This produces a dense value trace $v_i(t)$, where t denotes the raw frame index. We group held-out trajectories into three categories: clean successful trajectories, suboptimal successful trajectories with retry behavior, referred to as retry trajectories below, and failed trajectories. Global metrics evaluate whether the value model captures overall task progress and final task outcome, while local retry metrics evaluate whether the value trace exhibits the desired drop-and-rebound pattern around annotated retry events.

B.1 Global Metrics

Value-Order Correlation (VOC). VOC measures whether the predicted value increases with task progress on clean successful demonstrations. For each clean successful held-out trajectory, we compute the Spearman rank correlation between the 5 Hz frame index and the predicted value. We then report the average correlation over all clean successful trajectories. Recovery and failure trajectories are not included in VOC.

Success/Failure (S/F) Detection. S/F Detection measures whether the terminal value correctly predicts the final trajectory outcome. This metric is computed over all held-out successful and failed trajectories, where successful trajectories include both clean successes and retry trajectories. For each trajectory, we use the value score at the last evaluated frame as the trajectory-level score. We use a fixed threshold of 0.9 to distinguish successful and failed trajectories. The reported S/F Detection score is the classification accuracy against the ground-truth trajectory outcome. All value scores are normalized to the same range before computing this metric.

B.2 Local Retry Metrics

Local retry metrics evaluate whether a value model detects local mistakes and recovery around annotated retry events. These metrics are computed only on held-out recovery trajectories. We use a fixed evaluation radius $K = 30$ on the raw-frame timeline. This radius is used both for computing local value drops and for defining retry-centered evaluation windows. It is different from the pre-retry, retry-centered, and post-recovery sampling windows used for training in Sec. 3.2; here it is used only to evaluate whether the predicted value drops near the annotated retry keypoint and whether the retry region forms a local value minimum.

Let $\mathcal{R}_i$ denote the set of retry keypoints in trajectory i , and let $v_i(t)$ be the interpolated value at raw frame t . We compute a local drop score at each raw frame by comparing the current value with the maximum value within the preceding K -frame window:

$$D_i(t) = \max_{u \in [\max(0, t-K), t]} v_i(u) - v_i(t).$$

A large $D_i(t)$ indicates that the predicted value has recently dropped from a nearby local maximum. For a retry keypoint r , we use $[r - K, r + K]$ as its retry-centered evaluation window.

Drop AUC. Drop AUC measures whether value drops are concentrated near annotated retry events. For each retry keypoint, we use the maximum drop score within its retry-centered window as the positive-window score. Negative windows are sampled from regions at least $2K$ raw frames away from any retry keypoint, and their scores are computed in the same way. Let $\{(s_j, y_j)\}_{j=1}^N$ denote the resulting window-level examples, where s_j is the maximum drop score in the window and $y_j \in \{0, 1\}$ indicates

whether the window is retry-centered. We sort windows by s_j in descending order and compute the precision-recall AUC:

$$\text{Drop AUC} = \sum_{n=1}^N (R_n - R_{n-1}) P_n,$$

where P_n and R_n are the precision and recall after taking the top n windows. A higher Drop AUC indicates that large value drops are more localized around annotated retry events.

Drop Probability. Drop Probability measures how often annotated retries are accompanied by a salient value drop. For each trajectory, we define a trajectory-specific threshold

$$\eta_i = \max(\text{Quantile}_{0.9}\{D_i(t) : 0 \leq t \leq T_i\}, 0.01).$$

We then report the fraction of retry keypoints whose retry-centered window contains at least one drop score greater than η_i . Unlike Drop AUC, this metric does not use negative windows; it directly measures how consistently the model produces a detectable value drop around annotated retry events.

Pre > Retry and Post > Retry. These metrics test whether the annotated retry keypoint corresponds to a local low-value state. For each retry keypoint r , we use the same evaluation radius K defined above and compute the average values before and after the retry keypoint:

$$v_i^{\text{pre}}(r) = \frac{1}{K} \sum_{t=r-K}^{r-1} v_i(t), \quad v_i^{\text{post}}(r) = \frac{1}{K} \sum_{t=r+1}^{r+K} v_i(t).$$

Boundary cases where the full pre- or post-window is not available are skipped. **Pre > Retry** reports the fraction of retry events satisfying $v_i^{\text{pre}}(r) > v_i(r)$, while **Post > Retry** reports the fraction satisfying $v_i^{\text{post}}(r) > v_i(r)$. These metrics check whether the value decreases toward the retry keypoint and recovers after corrective behavior.

C Retry Annotation Analysis

C.1 Retry Keypoint Definition

A retry keypoint marks the start of correction after a local execution error, rather than the mistake onset or the completion of recovery. In our setting, retry events mainly capture localized but consequential execution errors that require immediate correction, such as misaligned grasps, missed grasps, or inaccurate placements.

We do not treat all execution errors as retry events. Instead, we only keep cases with a clear localized correction signal. We exclude long off-task error segments, such as grasping the wrong object and later completing the task, collisions with the environment, failed corrections that do not improve the state, and irreversible failures such as objects falling outside the reachable workspace. This keeps retry supervision tied to recoverable local execution errors rather than broad trajectory-level failures.

C.2 Annotation Cost and Consistency

We further evaluate the annotation cost and temporal consistency of retry-related labels. For annotation cost, three annotators independently label the value-model training trajectories for each task. We report the average annotation time per trajectory and its variation across annotators in Table 5. This measures the practical cost of obtaining sparse retry supervision. Table 5 shows that our sparse retry annotation can be obtained efficiently: even for the slowest annotator, labeling all 30 trajectories for a task takes no more than 30 minutes, corresponding to less than one minute per trajectory on average.

We next evaluate the temporal consistency of retry-related annotations. We measure consistency from both inter-annotator and intra-annotator perspectives. For inter-annotator consistency, three annotators independently label the same retry events. For intra-annotator consistency, the same annotator labels

Table 5: Annotation cost on the value-model training datasets. We report the total annotation time for each of three annotators in minutes and the per-trajectory time across annotators in seconds.

Task	# Retry/Succ./Fail Traj.	# Retry Keypts.	Ann. 1	Ann. 2	Ann. 3	Sec./Traj.
Pick up Spoon	20/5/5	20	10 min	15 min	8 min	22.0 ± 7.2
Stack Blocks	18/7/5	32	23 min	26 min	19 min	45.3 ± 7.0
Fold Towel	22/5/3	23	30 min	27 min	28 min	56.7 ± 3.1
Open Drawer	20/5/5	29	21 min	25 min	24 min	46.7 ± 4.2
Average	–	26.0	21.0 min	23.3 min	19.8 min	42.7 ± 14.1

Table 6: Temporal consistency of retry-related annotations. Inter-annotator errors are computed across three annotators, and intra-annotator errors are computed across three repeated passes by the same annotator. All values are in seconds.

Task	Inter-annotator error			Intra-annotator error		
	Pre	Retry	Post	Pre	Retry	Post
Pick up Spoon	0.42	0.17	0.46	0.24	0.09	0.27
Stack Blocks	0.55	0.14	0.72	0.16	0.09	0.20
Fold Towel	0.64	0.27	0.81	0.30	0.14	0.42
Open Drawer	0.43	0.19	0.63	0.21	0.16	0.31
Average	0.51	0.19	0.66	0.22	0.12	0.29

the same events three times. For each labeled timestep, let t_1, t_2, t_3 denote the three labeled timestamps, either from three annotators or three repeated passes. We compute the mean pairwise difference as

$$\text{error} = \frac{|t_1 - t_2| + |t_1 - t_3| + |t_2 - t_3|}{3}.$$

We report mean absolute timing errors averaged over retry events in Table 6. All errors are reported in seconds.

Table 6 shows that retry keypoints are temporally consistent across annotators. The average inter-annotator error for retry keypoints is only 0.19 seconds, which is less than one frame under our 5 Hz annotation rate and is acceptable for value-model training. In contrast, the auxiliary pre-retry and post-recovery timestamps are substantially less consistent, reflecting the ambiguity of when degradation begins or recovery is fully completed. This supports our design choice of using only the retry keypoint as supervision and applying a soft temporal window around it, rather than directly supervising the less reliable pre/post timestamps.

D Additional Experimental Results

D.1 Ablation Visualizations

We further provide representative value-curve visualizations to complement the ablation results in Sec. 4.3. Figure 6 shows that the drop-regression baseline without preference loss does not reliably align value drops with annotated retry events. Although this variant is trained with manually injected drops around retry windows, its predicted value curve often exhibits misplaced or weak drops on held-out trajectories. This suggests that directly shaping the absolute progress target does not necessarily teach the model to recognize the visual meaning of local execution errors. Instead, the model may fit the injected numeric pattern in the training data without learning robust visual cues for mistake states.

The visualizations also show that removing soft-window weighting or absolute calibration leads to less stable global progress estimates. Without soft-window weighting, retry-related supervision is applied uniformly across approximate retry neighborhoods, which can create sharper conflicts with the absolute progress target near window boundaries. As a result, the value curve becomes more oscillatory even on successful trajectories. Without absolute calibration, the model relies mainly on local retry-induced

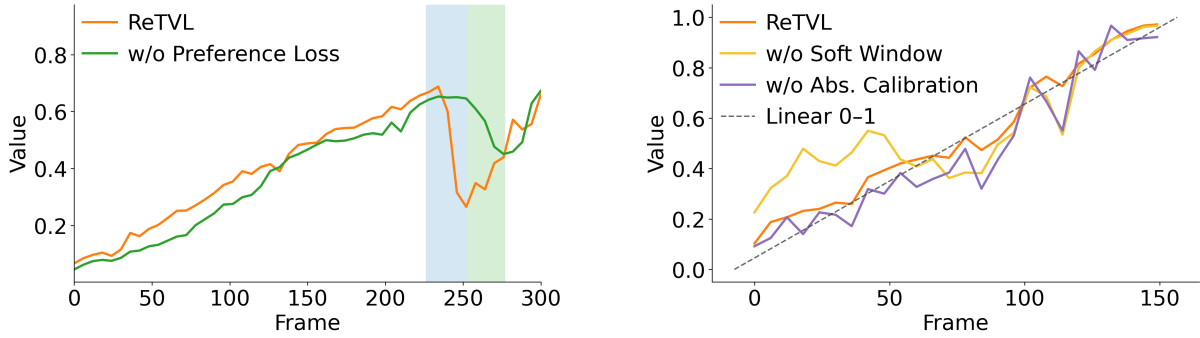


Figure 6: Representative value-curve visualizations for ablation variants on held-out trajectories.

Table 7: Chunk-weighting analysis averaged over four real-robot tasks.

Method	Success W. $\uparrow$	Post-retry W. $\uparrow$	Success Del. $\downarrow$	Strict-bad Ret. $\downarrow$
TOPReward	0.539	0.498	0.330	0.695
Robometer	0.845	0.432	0.085	0.896
RECAP-Value	0.716	0.438	0.150	0.743
ReTVL	0.769	0.636	0.146	0.206

preferences and loses a stable global progress anchor, which similarly increases value fluctuations and weakens the monotonic progress structure.

D.2 Analysis of Downstream Weights

We further report the chunk-weighting results of ReTVL and the baselines on the test set. This analysis complements the retry-centered value-shape metrics in Table 1, as it directly evaluates whether the resulting behavior-cloning weights preserve useful chunks and suppress harmful chunks. This analysis is less tied to the training loss and therefore provides a more direct view of how the learned values affect downstream policy learning.

We report four metrics in Table 7. *Success Weight* measures the average training weight assigned to chunks from successful trajectories. *Post-retry Weight* measures the average weight assigned to manually annotated recovery chunks after retry. *Success Deletion* measures the fraction of successful chunks that are incorrectly assigned zero weight, while *Strict-bad Retention* measures the fraction of manually annotated harmful chunks that are incorrectly retained with positive weight. Lower Success Deletion and Strict-bad Retention indicate better filtering behavior.

Table 7 shows that ReTVL achieves the most favorable trade-off between preserving useful chunks and suppressing harmful ones. Robometer assigns the highest weights to successful chunks and has the lowest Success Deletion rate, but it also retains most strict-bad chunks, with a Strict-bad Retention rate of 0.896. In contrast, ReTVL assigns the highest weights to post-retry recovery chunks while reducing Strict-bad Retention to 0.206, substantially lower than all baselines. These results support the central role of retry-supervised value learning: ReTVL does not merely suppress all retry-related data, but learns a more selective weighting signal that preserves useful recovery behavior and downweights harmful mistake segments.